

PORTAL DE HERRAMIENTAS DE SOPORTE EPEM

Guia de aprendizaje paso a paso

Backend NestJS + datos JSON

Modulo 1: Estructura, datos y primer modulo

Septiembre 2026

. INDICE

1. Concepto del proyecto
2. Arquitectura general (backend + frontend + datos)
3. Los archivos JSON como base de datos temporal
4. Estructura de un modulo NestJS (Controller, Service, Module)
5. El Service: procedimientos.service.ts (explicacion linea por linea)
6. El Controller: procedimientos.controller.ts (explicacion linea por linea)
7. El Module: procedimientos.module.ts
8. Registrar el modulo en AppModule
9. Errores comunes y sus soluciones
10. Endpoints que funcionan
11. Conceptos clave para recordar

1. Concepto del proyecto

El Portal de Herramientas de Soporte EPEM es una aplicación interna con diferentes módulos para que el equipo de soporte trabaje mejor con el sistema EPEM. NO se conecta a la base de datos de EPEM: los datos viven en archivos locales JSON.

Modulos del portal:

- Base de conocimientos
- Buscar errores
- Generar reporte
- Procedimientos
- Herramientas
- Documentación

CONCEPTO CLAVE: La ventaja: no estamos desarrollando otra parte del sistema EPEM, sino una herramienta propia para que el equipo

2. Arquitectura general

El proyecto tiene dos partes que se comunican entre si:

```
PORTAL_DE_HERRAMIENTAS/  
  backend/      <- NestJS (API REST)  
    data/       <- archivos JSON (base de datos temporal)  
    src/        <- código del backend  
  frontend/    <- React + Vite (interfaz de usuario)
```

Flujo de una petición:

```
Frontend --GET /procedimientos--> Controller --> Service (lee JSON) --> JSON al Frontend
```

El frontend consume el backend como una API REST normal. El frontend NO sabe que los datos vienen de archivos JSON: para él, es una API como cualquier otra.

3. Los archivos JSON como base de datos temporal

Usamos JSON en lugar de una base de datos real por tres motivos:

1. No necesitamos instalar nada (no MySQL, no PostgreSQL, no Docker).
2. Los datos son de solo lectura al principio: el soporte los lee, no los edita desde el portal.
3. Si en el futuro queremos pasar a una BD real, la estructura ya esta definida: cada JSON se convierte en una tabla.

Los 4 archivos creados en backend/data/:

```
errores.json          <- Errores comunes con codigo, descripcion y solucion
procedimientos.json  <- Pasos paso a paso para resolver problemas
soluciones.json       <- Respuestas rapidas tipo FAQ
configuracion.json    <- Datos estaticos (grupos, estados, modulos)
```

Cada archivo es una 'tabla' de nuestra base de datos temporal. El backend los lee del disco y los devuelve al frontend.

4. Estructura de un modulo NestJS

En NestJS, un modulo se compone de 3 partes:

Controller	<- Recibe las peticiones HTTP (GET, POST, etc.)
Service	<- Contiene la logica de negocio (leer archivos, filtrar)
Module	<- Agrupa Controller + Service y lo registra en la app

CONCEPTO CLAVE: El Service es el 'cerebro' (logica). El Controller es el 'mesero' (recibe pedidos HTTP y delega al Service). El Module es

5. El Service: procedimientos.service.ts

El Service contiene la logica de negocio. No sabe nada de HTTP ni de URLs. Solo sabe donde estan los datos, como leerlos y como filtrarlos.

Codigo completo:

```
import { Injectable } from '@nestjs/common';
import * as fs from 'fs';
import { fileURLToPath } from 'url';
import * as path from 'path';

export interface Paso {
  orden: number;
  descripcion: string;
}

export interface Procedimiento {
  id: number;
  titulo: string;
  pasos: Paso[];
  modulo: string;
  nivel: string;
  tiempo_estimado: string;
}

@Injectable()
export class ProcedimientosService {
  private readonly dataPath: string;

  constructor() {
    const __filename = fileURLToPath(import.meta.url);
    const __dirname = path.dirname(__filename);
    this.dataPath = path.join(__dirname, '..', '..', 'data', 'procedimientos.json');
  }

  findAll(): Procedimiento[] {
    const rawData = fs.readFileSync(this.dataPath, 'utf-8');
    const procedimientos: Procedimiento[] = JSON.parse(rawData);
    return procedimientos;
  }

  findOne(id: number): Procedimiento | undefined {
    const todos = this.findAll();
    return todos.find((p) => p.id === id);
  }

  findByModulo(modulo: string): Procedimiento[] {
    const todos = this.findAll();
    return todos.filter((p) => p.modulo === modulo);
  }
}
```

}

Explicacion linea por linea

```
import { Injectable } from '@nestjs/common';
```

Importa el decorador `@Injectable`. Le dice a Nest: 'esta clase puede ser usada por otras clases'. Sin esto, el Controller no podria pedirle datos al Service.

```
import * as fs from 'fs'; y import * as path from 'path';
```

Modulos nativos de Node.js: `fs` (file system) para leer archivos del disco, y `path` para construir rutas de forma segura (funciona en Windows y Linux).

```
interface Paso { orden: number; descripcion: string; }
```

Interfaces de TypeScript. No generan codigo JavaScript, solo le dicen al compilador la forma que deben tener los datos. Si escribis mal una propiedad, TypeScript te avisa antes de ejecutar.

```
@Injectable()
```

Decorador que registra esta clase en el contenedor de dependencias de NestJS.

```
private readonly dataPath: string;
```

Propiedad privada que guarda la ruta al archivo JSON. 'readonly' significa que solo se asigna una vez (en el constructor).

```
constructor() { const __filename = fileURLToPath(import.meta.url); ... }
```

En ES modules no existe `__dirname` (como en CommonJS). Usamos `import.meta.url` y `fileURLToPath` para obtener la ruta del archivo actual, y `path.dirname` para obtener la carpeta.

```
findAll(): Procedimiento[] { ... }
```

`fs.readFileSync` lee el archivo sincronicamente (espera a que termine). `JSON.parse` convierte el texto JSON (string) en objetos JavaScript reales. Devuelve un array de Procedimientos.

```
findOne(id: number): Procedimiento | undefined { ... }
```

Usa el metodo nativo `.find()` que devuelve el PRIMER elemento que cumpla la condicion. '`| undefined`' le dice a TypeScript que puede devolver un Procedimiento o undefined si no existe.

```
findByModulo(modulo: string): Procedimiento[] { ... }
```

Usa `.filter()` que devuelve TODOS los elementos que cumplan la condicion (no solo el primero).

6. El Controller: procedimientos.controller.ts

El Controller es la puerta de entrada. Recibe las peticiones HTTP del navegador y decide que Service llamar, que metodo usar y que codigo HTTP devolver.

Codigo completo:

```
import { Controller, Get, Param, Query } from '@nestjs/common';
import { ProcedimientosService, Procedimiento } from '../procedimientos.service.js';

@Controller('procedimientos')
export class ProcedimientosController {
  constructor(private readonly procedimientosService: ProcedimientosService) {}

  @Get()
  findAll(@Query('modulo') modulo?: string): Procedimiento[] {
    if (modulo) {
      return this.procedimientosService.findByModulo(modulo);
    }
    return this.procedimientosService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id') id: string): Procedimiento | { statusCode: number; message: string } {
    const procedimiento = this.procedimientosService.findOne(Number(id));
    if (!procedimiento) {
      return { statusCode: 404, message: 'Procedimiento no encontrado' };
    }
    return procedimiento;
  }
}
```

Explicacion linea por linea

```
import { Controller, Get, Param, Query } from '@nestjs/common';
```

Importa los decoradores: `@Controller` (define la ruta base), `@Get` (responde a GET), `@Param` (extrae valores de la URL), `@Query` (extrae query params).

```
@Controller('procedimientos')
```

Todas las rutas de esta clase empiezan con `/procedimientos`. Ej: `/procedimientos`, `/procedimientos/1`.

```
constructor(private readonly procedimientosService: ProcedimientosService) {}
```

Inyeccion de dependencias. NestJS automaticamente crea una instancia de `ProcedimientosService` y la pasa al constructor. No hace falta hacer `'new ProcedimientosService()'`.

```
@Get() findAll(@Query('modulo') modulo?: string)
```

`@Get()` sin parametros responde a GET `/procedimientos`. `@Query('modulo')` busca `?modulo=...` en la URL. El `'?'` significa que es opcional. Si la URL es `/procedimientos?modulo=Comisiones`, modulo vale `'Comisiones'`.

```
if (modulo) { return this.procedimientosService.findByModulo(modulo); } return this.procedimientosService.findAll();
```

Si vino el query param modulo, filtra. Si no, devuelve todos.

```
@Get(':id') findOne(@Param('id') id: string)
```

`@Get(':id')` responde a GET `/procedimientos/1`, `/procedimientos/2`, etc. `@Param('id')` captura el `'1'` de la URL como string.

```
const procedimiento = this.procedimientosService.findOne(Number(id));
```

Convierte el string `'1'` al numero 1 (`Number(id)`) y busca en el Service. Si no existe, devuelve un objeto con `statusCode: 404`.

7. El Module: procedimientos.module.ts

El Module es el registro. Le dice a NestJS que este modulo tiene un Controller y un Service, y los hace disponibles para el resto de la aplicacion.

Codigo completo:

```
import { Module } from '@nestjs/common';
import { ProcedimientosController } from './procedimientos.controller.js';
import { ProcedimientosService } from './procedimientos.service.js';

@Module({
  controllers: [ProcedimientosController],
  providers: [ProcedimientosService],
})
export class ProcedimientosModule {}
```

controllers: lista de controladores que manejan rutas HTTP.

providers: lista de servicios que pueden ser inyectados.

El Module es simple: solo registra el Controller y el Service.

8. Registrar el modulo en AppModule

Sin registrar el ProcedimientosModule en el AppModule raiz, NestJS no sabe que existe. Hay que importarlo.

Codigo de app.module.ts:

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller.js';
import { AppService } from './app.service.js';
import { ProcedimientosModule } from './procedimientos/procedimientos.module.js';

@Module({
  imports: [ProcedimientosModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

NOTA: Tambien se elimino el modulo ObserveModule (telemetria de NestJS) que venia por defecto y no necesitamos. Y en main.ts se q

9. Errores comunes y sus soluciones

Error 1: imports sin extension .js

Sintoma: 'Cannot find module ./procedimientos.service'.

Causa: el tsconfig usa moduleResolution 'nodenext', que exige la extension .js en los imports relativos.

Solucion: escribir './procedimientos.service.js' en lugar de './procedimientos.service'.

Error 2: __dirname is not defined

Sintoma: 'ReferenceError: __dirname is not defined'.

Causa: en ES modules no existe __dirname (es de CommonJS).

Solucion: usar import.meta.url + fileURLToPath + path.dirname para obtener la carpeta actual.

Error 3: Return type cannot be named

Sintoma: 'Return type of public method ... cannot be named'.

Causa: las interfaces no estaban exportadas, así que el Controller no podía referenciarlas.

Solucion: agregar 'export' delante de las interfaces (export interface Procedimiento).

Error 4: EADDRINUSE (puerto ocupado)

Sintoma: 'Error: listen EADDRINUSE: address already in use :::3000'.

Causa: ya hay un proceso usando el puerto 3000.

Solucion: levantar en otro puerto con PORT=3001 npm run start:dev, o matar el proceso anterior.

10. Endpoints que funcionan

Con el backend corriendo (npm run start:dev), estos endpoints responden:

```
GET http://localhost:3001/procedimientos
-> Todos los procedimientos

GET http://localhost:3001/procedimientos?modulo=SIFEN
-> Filtra por modulo

GET http://localhost:3001/procedimientos/1
-> Un procedimiento por ID
```

Para probar desde la terminal:

```
curl -s http://localhost:3001/procedimientos
```

11. Conceptos clave para recordar

Service: contiene la logica de negocio, no sabe de HTTP.

Controller: puerta de entrada HTTP, delega al Service.

Module: registro de componentes en NestJS.

Inyeccion de dependencias: Nest crea automaticamente las instancias y las pasa al constructor.

Interfaces TypeScript: definen la forma de los datos para evitar errores en tiempo de compilacion.

ES modules: usan `import.meta.url` en vez de `__dirname`.

JSON como BD temporal: cada archivo es una 'tabla'; el backend los lee del disco.

Decoradores: `@Controller`, `@Get`, `@Param`, `@Query`, `@Injectable`, `@Module`. Son funciones que agregan metadatos a clases y metodos.

Metodos nativos de arrays: `.find()` devuelve el primero que cumpla, `.filter()` devuelve todos los que cumplan.